

Cours IoT – Séance 4

Communication série Arduino ↔ Node-RED ↔ MariaDB

Objectifs pédagogiques

À la fin de cette séance, vous serez capables de :

1. Lire des données provenant d'un Arduino réel via le port série sur la Raspberry Pi.
2. Décoder (parser) une trame reçue au format JSON.
3. Insérer automatiquement ces données dans la base MariaDB déjà configurée.
4. Vérifier le flux complet : Capteur Arduino → Raspberry Pi → Node-RED → MariaDB → Grafana (prochaine séance).

Matériel nécessaire

- Raspberry Pi avec Docker + Node-RED + MariaDB déjà installés (séance 1)
- Arduino avec capteur (DHT22, LDR, etc.) programmé pour envoyer une trame JSON (séance 2)
- Câble USB entre Arduino et Raspberry Pi
- Accès SSH à la Raspberry Pi

Étape 1 – Vérification de la communication série

1. Identifier le port série de l'Arduino :

Après vous êtes connecté en ssh à votre raspberry, tapez les commandes suivantes pour lister les périphériques series.

- `ls /dev/tty*`
- `dmesg | grep tty`

```

remi@Remi:~$ ls /dev/tty*
/dev/tty /dev/tty19 /dev/tty3 /dev/tty40 /dev/tty51 /dev/tty62
/dev/tty0 /dev/tty2 /dev/tty30 /dev/tty41 /dev/tty52 /dev/tty63
/dev/tty1 /dev/tty20 /dev/tty31 /dev/tty42 /dev/tty53 /dev/tty7
/dev/tty10 /dev/tty21 /dev/tty32 /dev/tty43 /dev/tty54 /dev/tty8
/dev/tty11 /dev/tty22 /dev/tty33 /dev/tty44 /dev/tty55 /dev/tty9
/dev/tty12 /dev/tty23 /dev/tty34 /dev/tty45 /dev/tty56 /dev/ttyprintk
/dev/tty13 /dev/tty24 /dev/tty35 /dev/tty46 /dev/tty57 /dev/ttyUSB0
/dev/tty14 /dev/tty25 /dev/tty36 /dev/tty47 /dev/tty58
/dev/tty15 /dev/tty26 /dev/tty37 /dev/tty48 /dev/tty59
/dev/tty16 /dev/tty27 /dev/tty38 /dev/tty49 /dev/tty6
/dev/tty17 /dev/tty28 /dev/tty39 /dev/tty5 /dev/tty60
/dev/tty18 /dev/tty29 /dev/tty4 /dev/tty50 /dev/tty61
remi@Remi:~$ dmesg | grep tty
[ 0.000000] Kernel command line: coherent_pool=1M 8250.nr_ua
rts=0 snd_bcm2835.enable_headphones=0 cgroup_disable=memory
snd_bcm2835.enable_headphones=1 snd_bcm2835.enable_hdmi=1
vc_mem.mem_base=0x3ec00000 vc_mem.mem_size=0x40000000 console=
ttyS0,115200 console=tty1 root=PARTUUID=42ec2a51-02 rootfstype=
ext4 fsck.repair=yes rootwait quiet splash plymouth.ignore-serial-
consoles cfg80211.ieee80211_regdom=BE
[ 0.000414] printk: legacy console [tty1] enabled
[ 4.441067] 3f201000.serial: ttyAMA1 at MMIO 0x3f201000 (irq =
99, base_baud = 0) is a PL011 rev2
[ 4.442614] serial serial0: tty port ttyAMA1 registered
[ 15.620011] systemd[1]: Created slice system-getty.slice - Slice
/system/getty.slice.
[ 18.937252] usb 1-1.1.2: ch341-uart converter now attached to
ttyUSB0
remi@Remi:~$

```

2. Vérifier les données envoyées :

- cat /dev/ttyUSB0 (Attention, le nom de votre port USB peut être différent de USB0, utilisez le votre)

```

remi@Remi:~$ cat /dev/ttyUSB0
PuTTYPuTTY

```

Le resultat est potentiellement illisible si le baude rate de lecture n'est pas le bon, ici 115200.

Pour cela, minicom pour afficher les trames dans le bon format. Pour cela, installer minicom via la commande :

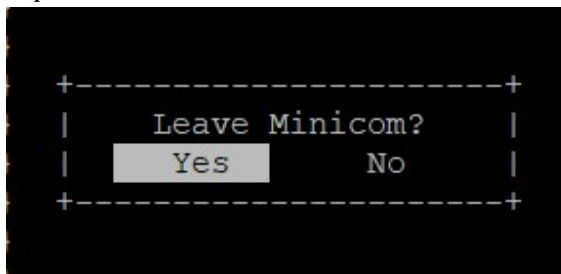
```
sudo apt install minicom
```

Puis lancer minicom sur votre port usb et avec le bon baude rate via la commande :

```
sudo minicom -b 115200 -o -D /dev/ttyUSB0
```

```
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 18000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 20000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 22000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 24000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 26000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 28000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 30000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 32000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 34000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 36000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 38000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 40000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 42000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 44000 }
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 46000 }
```

Pour quitter l'écran : ctrl A + X



Nous utilisons Node-Red dans un conteneur qui tourne à part du système comme une machine virtuelle. Il faut alors donner à ce conteneur l'accès au port USB sur lequel est connecté l'arduino. Pour cela, il faut ajouter ce port dans le fichier docker-compose.yml.

Dans la partie du fichier qui concerne Node-Red, ajouter:

```
devices:
  - /dev/ttyUSB0:/dev/ttyUSB0
group_add:
  - dialout
environment:
  - TZ=Europe/Brussels
```

Une fois le fichier à jour, relancer le docker:

```
docker compose down
docker compose up -d
```

🔗 Étape 2 – Lecture du port série dans Node-RED

1) Ajouter et configurer le nœud Serial in

1. Faites glisser un nœud « serial in » dans le flow.
2. Double-cliquez et configurez :

- Serial port : /dev/ttyUSB0
- Baud rate : 115200
- Data bits : 8 | Parity : none | Stop bits : 1
- Output : a single message per line (newline = \n)

3. Ajoutez un nœud Debug (« raw ») relié au Serial in et déployez. Vous devez voir les lignes JSON s'afficher.

2) Parser la trame JSON

1. Ajoutez un nœud « json » et reliez-le au Serial in.
2. Ajoutez un nœud Debug (« parsed ») après le nœud json pour vérifier l'objet JavaScript résultant.

Exemple de trame attendue :

```
{ "sensor": "AM2302", "ok": true, "temp_c": 24.8, "temp_f": 76.6, "hum": 2.0, "ts_ms": 46000 }
```

3) Construire la requête SQL (paramétrée)

Ajoutez un nœud Function avec le code suivant (sécurisé avec paramètres ?) :

```
var p = msg.payload;
if (typeof p !== "object") { return null; }
msg.topic = "INSERT INTO capteurs (device_id, capteur, valeur, unite, ts) VALUES (?, ?, ?, ?, NOW())";
msg.payload = [ (p.sensor||"sensor").toString(), Number(p.temp_c)||null,
Number(p.temp_f)||null, Number(p.hum)||null, Number(p.ts_ms)||null ];
return msg;
```



Étape 3 – Insertion automatique dans MariaDB

Utilisez un module MariaDB comme celui de la séance 3 pour insérer les données reçues en série de l'arduino vers votre base de données.



Étape 4 – Vérification dans la base de données

```
docker exec -it mariadb mariadb-u root -p
USE iot_lab;
SELECT * FROM capteurs ORDER BY ts DESC LIMIT 5;
```